AI Product Matching System

Database System — Full Explanation

This document explains how data is stored and organized across the project's two-database architecture: a PostgreSQL database for structured, transactional data, and a Qdrant vector database for semantic similarity search. It covers the full schema, why each table and column exists, how the two databases stay in sync, and how the design evolved across the project's eight development phases.

Companion documents in the project: **DATABASE.md** (schema reference), **ARCHITECTURE.md** (full design rationale for every phase), **README.md** (how to run it).

1. Why Two Databases

The system stores data in two fundamentally different places because it needs to answer two fundamentally different kinds of questions:

- **"What are the exact facts about this product?"** — its price, quantity, upload history, who confirmed the match, when. This is structured, relational data with clear identity and row-by-row traceability: **PostgreSQL**.
- **"Which products in the catalog mean roughly the same thing as this one?"** — a fuzzy, semantic question that has no exact answer, only a ranked list of similarities. This is what vector databases are built for: **Qdrant**.

Trying to answer the second question with a relational database alone would mean either exact text matching (which breaks the moment two people describe the same product differently) or expensive full-text search that still misses semantic similarity. Trying to store transactional records in a vector database would give up the reliability, constraints, and query flexibility SQL provides for that data. Keeping them separate — connected only by shared product IDs — lets each database do the one job it is actually good at.

How they connect

Every product row in PostgreSQL has a UUID primary key. When the master catalog is indexed, each non-header product's text is embedded and stored as one point in Qdrant, with that same UUID attached as payload metadata. Search results from Qdrant come back as a list of UUIDs and similarity scores; those UUIDs are then used to look the full product record back up in PostgreSQL. Qdrant never stores prices, quantities, or feedback — only enough to find the ID and explain the score. Postgres never stores embeddings.

2. PostgreSQL Schema

Six tables, in the order data flows through the system: upload → catalog rows → matches → feedback.

2.1 uploads

One row per Excel file processed (either a master catalog or a destination file). Tracks ingestion progress and any per-row errors, so a bad file never silently fails and never crashes the whole batch.

Column	Type	Notes
id	UUID (PK)	
filename	text	original uploaded filename
upload_type	text	"master" or "destination"
sheet_name	text	which sheet within the Excel file was ingested
status	text	pending / processing / done / failed
total_rows	int	rows detected in the sheet

Column	Type	Notes
processed_rows	int	rows successfully ingested
skipped_rows	int	malformed rows skipped, not fatal
error_report	jsonb	list of {row_number, reason} for every skipped row
created_at	timestamptz	

2.2 master_products

One row per product in the master catalog (the large, canonical price list). Columns come from mapping whatever headers the source spreadsheet actually used (e.g. "[redacted]", "[redacted]", "[redacted]") onto this fixed set of fields.

Column	Type	Notes
id	UUID (PK)	
upload_id	UUID (FK)	→ uploads.id
source_row	int	original Excel row number, for traceability
external_id	text	product code / SKU, e.g. 521-101-0131-0001
product_name	text	mapped from the name column
normalized_name	text	lowercased, whitespace-collapsed, punctuation-normalized
description	text	
unit	text	unit of measure, e.g. "[redacted]."
price	numeric	
freight_class	text	shipping classification, if present
gross_weight_kg	numeric	
is_group_header	boolean	true when unit and price are both empty — a catalog section header, not a purchasable product
raw_data	jsonb	the complete original row, untouched
created_at	timestamptz	

2.3 destination_products

One row per product in a destination file — the list of items that need to be matched against the master catalog.

Column	Type	Notes
id	UUID (PK)	
upload_id	UUID (FK)	→ uploads.id
source_row	int	original Excel row number, for traceability

Column	Type	Notes
external_id	text	often blank in real destination files
product_name	text	
normalized_name	text	
description	text	
quantity	numeric	
price	numeric	
status	text	pending / matched / no_match
uncertainty_margin	numeric	gap between the top-2 candidate scores; NULL until the prioritization step runs; smaller = more ambiguous = reviewed first
raw_data	jsonb	the complete original row, untouched
created_at	timestamptz	

2.4 matches

One row per confirmed pairing between a destination product and a master product — whichever method produced it: a human clicking "Confirm," or the system auto-accepting a high-confidence or exact match.

Column	Type	Notes
id	UUID (PK)	
destination_product_id	UUID (FK)	→ destination_products.id
master_product_id	UUID (FK)	→ master_products.id
confidence	numeric	the matching score behind this decision
match_type	text	e.g. user_selected, auto_accepted
method	text	exact_match / auto_threshold / hybrid_top3 / manual_search
is_confirmed	boolean	
created_at	timestamptz	

2.5 feedback

One row for *every* decision made about a destination product, whether or not it resulted in a match — including the complete set of candidates that were shown at the time, so nothing about a past decision is ever lost. This table is the project's memory: it is what a future machine-learning model would be trained on.

Column	Type	Notes
id	UUID (PK)	
destination_product_id	UUID (FK)	→ destination_products.id
selected_master_product_id	UUID (FK), nullable	→ master_products.id; NULL when the decision was "no match"

Column	Type	Notes
decision_type	text	user_selected / manual_search_selected / no_match / auto_accepted / auto_rejected / user_rejected (reserved)
candidate_data	jsonb	destination name, selected product, and the full ranked candidate list with scores that was shown
created_at	timestamptz	

2.6 match_candidates (reserved, not yet populated)

Defined in the schema for forward compatibility — a normalized, per-candidate table (rank, vector/keyword/fuzzy/reranker/final scores) that a later phase could use instead of storing the candidate list as JSON inside **feedback**. Storing candidates as JSON has been sufficient so far; this table exists so the schema doesn't need to change later if that stops being true.

Column	Type	Notes
id	UUID (PK)	
destination_product_id	UUID (FK)	→ destination_products.id
master_product_id	UUID (FK)	→ master_products.id
rank	int	1, 2, 3, ... within the candidate list
vector_score	numeric	Qdrant semantic similarity score
keyword_score	numeric	BM25 keyword-overlap score
fuzzy_score	numeric	fuzzy string-similarity score
reranker_score	numeric	score after reranking
final_score	numeric	the blended score actually used to rank the candidate

3. Key Design Decisions

3.1 Every row keeps its original data (raw_data)

Both **master_products** and **destination_products** store the complete original spreadsheet row as JSON in a **raw_data** column, in addition to the cleaned/mapped fields. This exists so that every match can always be traced back to exactly what was in the source Excel file — regardless of how the column-mapping or normalization logic changes in the future. Nothing about the original file is ever discarded, even for rows that fail to parse cleanly.

3.2 Catalog section headers are kept, not deleted (is_group_header)

Real master catalogs mix real, priced products with section headers (e.g. "521-101-0100 — ■■■■■■■■■■■■ ■■■■■■■■", a category label with no unit or price). Rather than filtering these out during ingestion — which would silently lose part of the source file — every row is kept, and rows with no unit and no price are flagged **is_group_header = true**. Search and matching logic then explicitly excludes flagged rows from candidate results, but the data itself, and the category structure it represents, is preserved for later use.

3.3 Column mapping is alias-based, not hardcoded

Different source files name the same column differently — "■■■■■■■■■■■■■", "■■■■■■■■■", "■■■■■■", "Product Name" all mean the same thing. Rather than requiring one exact column layout, an alias dictionary maps any of these onto one canonical field before the row is stored. This keeps the schema stable even though real-world spreadsheets are inconsistent.

3.4 Every decision is recorded, not just the outcome

It would be simpler to just update **destination_products.status** and move on. Instead, every confirm, reject, and automatic decision also writes a full **feedback** row, including the exact list of candidates and scores that were shown at the time. The practical reason: this is what turns the system's history into a labeled dataset — confirmed matches become positive training examples, and every candidate that was shown but *not* picked becomes a hard negative example, which is a far more useful signal than an unrelated random product would be.

3.5 Automatic decisions get their own labels

The system distinguishes *why* a decision was made, not just what the decision was. **auto_accepted** and **auto_rejected** are separate from a human's **user_selected** or **no_match**, so it's always possible to answer "how many of these matches did a human actually look at?" — which matters both for auditing and for deciding whether the automation thresholds are calibrated sensibly.

3.6 Uncertainty is a stored column, not a live computation

Deciding which pending item to show a reviewer next by "how uncertain is the system about this one" would, if computed live, mean re-running the full search pipeline for every pending item on every page load — too

expensive at scale. Instead, **uncertainty_margin** is computed once in a batch step and stored, so ordering the review queue by ambiguity is a fast, ordinary column sort.

3.7 SQLite as a zero-dependency fallback

The application code targets PostgreSQL, but the same code path also runs against SQLite by changing one configuration value. This exists purely for low-friction local development and testing — anyone can run the full pipeline without installing or configuring a database server first — and is not intended as a production choice.

4. The Vector Database (Qdrant)

Qdrant stores exactly one thing: a numeric embedding (a list of numbers representing the meaning of a product's text) for every non-header master product, plus a small amount of payload metadata used for filtering and for looking the full record back up in PostgreSQL.

4.1 One shared collection, not one per category

It would be tempting to build a separate small vector index per product category (furniture, toys, medical equipment, ...). This project deliberately does not do that: it keeps a single collection named **products** for the entire master catalog, and relies on metadata filters (rather than physically separate databases) whenever a search needs to be scoped. A single collection is far easier to keep consistent as the catalog grows and changes, and avoids the operational overhead of managing dozens or hundreds of small, easily-out-of-sync indexes.

4.2 What gets embedded

Each master product's embedding is generated from its normalized name plus description — not just the bare product name — so that semantic search has more real signal to work with. The same text-construction logic is applied to destination products at query time, so both sides of a comparison go through an identical pipeline before their vectors are compared.

4.3 Two interchangeable ways to run it

The same code can talk to Qdrant in two different modes, selected by configuration alone:

- **Embedded / local mode** — Qdrant runs as an on-disk engine inside the application process itself, no separate server required. Used for quick local development and testing.
- **Server mode** — a real Qdrant server (the Docker Compose service in this project) that the application connects to over HTTP. This is the mode intended for actual use, and is what the single-command Docker setup starts automatically.

Both modes speak the exact same client API, so switching between them never requires touching the search or matching code — only a connection setting.

4.4 What is deliberately not stored in Qdrant

Prices, quantities, upload history, and every piece of feedback stay in PostgreSQL only. Qdrant holds just enough — the embedding and a product ID — to answer "which products are semantically close to this text," and nothing more. This keeps the vector database small, fast, and free of any data-consistency responsibility; PostgreSQL remains the single source of truth for anything that matters for correctness or auditing.

5. How Data Moves Through the System

The two databases are touched at different points in the same overall pipeline:

Step	PostgreSQL	Qdrant
1. Upload master catalog	New uploads row; one master_products row per catalog line	— (not yet)
2. Build the search index	Reads master_products	One embedding point written per non-header product
3. Upload destination file	New uploads row; one destination_products row per line	— (not yet)
4. Search / match a destination product	Reads the destination_products row for its text	Queried for the closest master-product vectors
5. Human confirms / rejects, or auto-match runs	Writes matches + feedback; updates destination_products.status	not touched
6. Prioritize by uncertainty	Writes destination_products.uncertainty_margin	Queried once per pending product to compute the margin
7. Train a matching model	Reads feedback to build a labeled training set	Re-queried to recover per-candidate scores

Notably, PostgreSQL is written to at every step; Qdrant is only written to once, when the search index is (re)built from the master catalog, and is read from repeatedly afterward. This asymmetry reflects their different roles: Postgres is the transactional record of everything that happened, while Qdrant is a derived, rebuildable search index.

5.1 Rebuilding the index

Because Qdrant's contents are fully derived from master_products, the entire vector index can be thrown away and rebuilt at any time without losing information — it is a cache-like index over PostgreSQL, not an independent source of truth. The application takes advantage of this: it automatically rebuilds the index on startup if a master catalog is already present, so restarting the backend does not require re-uploading anything.

6. Current State and Known Limitations

A few things worth knowing about the current state of this database system, stated plainly rather than glossed over:

- **match_candidates is defined but unused.** Candidate lists are currently stored as JSON inside feedback.candidate_data, which has been sufficient so far; the normalized table exists for if that changes.
- **destination_products has no unit column**, even though master_products does and the column-mapping logic already recognizes a "unit" field on upload. This means unit-of-measure can't currently be used as a matching signal for destination products — a known, fixable gap rather than an intentional design choice.
- **No category or brand columns exist yet** on either product table. Attribute extraction and category classification were never built, which is the main reason the matching score and any future machine-learning features are based only on text-similarity signals (keyword, fuzzy, semantic) plus price — not on structured attributes like category, brand, or model.
- **SQLite is a development convenience, not a production target.** The schema and code work identically against both, but PostgreSQL is the intended database for real use.

For the complete, always-current schema reference, see **DATABASE.md** in the project root. For the full reasoning behind every phase of the system (not just the database), see **ARCHITECTURE.md**.